

Why this architecture fits Chairly

The reasoning, tradeoffs, alternatives, and reassessment triggers behind the proposed multi-tenant booking platform.

STATUS	SCOPE	SOURCE
Proposed	MVP foundation	PRD + Architecture + AGENTS

EXECUTIVE PERSPECTIVE

The architecture follows the risk, not the fashion

Chairly is not primarily a content website. It is a scheduling system where a data leak or double-booking damages trust immediately. The architecture therefore concentrates complexity around tenant isolation, time, concurrency, auditability, and recoverable side effects - while keeping deployment and team coordination simple.

The governing idea

Start with the smallest architecture that can enforce the hardest business invariants. Keep vendor choices replaceable, but make security and booking correctness non-negotiable.

1	Correctness first	The database is the final authority for tenant boundaries and staff-slot ownership.
2	One product, one model	Public pages, dashboard, and booking actions share one domain model during the MVP.
3	Reversible edges	Authentication, storage, email, and hosting sit behind adapters so vendors can change.
4	Measured evolution	Services are extracted only when load, team ownership, or deployment isolation proves the need.

This is a deliberately conservative design: sophisticated where failure is costly, plain where simplicity accelerates learning.

DECISION LENS

Product constraints translated into architecture

Each major choice traces to an explicit product pressure. This prevents architecture from becoming a catalogue of preferred tools disconnected from the business.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Many independent businesses	Every tenant needs a branded page and private workspace without separate deployments.	A deployment per tenant raises operating cost and makes upgrades inconsistent.
Booking requests require confirmation	Pending and confirmed states both reserve capacity and must be audited.	A simple calendar event model cannot express the request lifecycle reliably.
Customers may be anonymous	Public booking must be low-friction while management actions remain secure.	Mandatory customer accounts would reduce conversion and add identity complexity.
Owners are small operators	Managed infrastructure and one deployable application reduce operational burden.	Microservices would shift effort from customer value to platform coordination.
Double-booking is unacceptable	The final overlap rule must live in a transactional database constraint.	UI-only availability checks lose races under simultaneous requests.
Notifications can fail	Bookings commit independently; delivery is retried from an outbox.	Sending email inside the transaction couples core success to a third party.
Future payments and custom domains	The model preserves extension points without implementing speculative features.	Building those features now would widen scope before product-market evidence.

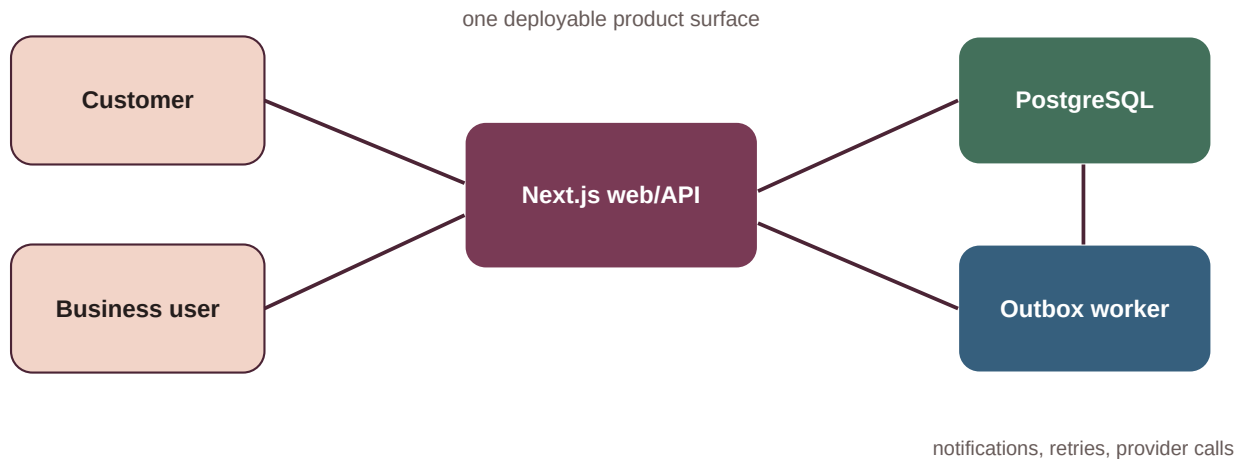
What this excludes

The architecture does not promise infinite scale, instant global consistency, or zero vendor dependence. It promises a credible, secure MVP foundation with clear signals for when to evolve.

DECISION 01

A modular monolith before microservices

One Next.js application and one PostgreSQL system of record provide the fewest moving parts that still support strong internal boundaries.



Why it was chosen

- The initial product journeys cross identity, catalog, scheduling, appointments, notifications, and audit. Keeping them in one process makes transactions and iteration straightforward.
- A small team can deploy, observe, and debug one product surface without distributed tracing being required just to understand a booking.
- Package boundaries preserve future extraction options: domain logic does not depend on React, Next.js, database drivers, or vendor SDKs.

Tradeoff accepted

A monolith can accumulate coupling if boundaries are ignored. AGENTS.md makes dependency direction, server-only database access, and module ownership reviewable rules rather than informal preferences.

Revisit when independently scaling or deploying one module produces a measured operational benefit, or when separate teams require clear service ownership. The notification worker is the first likely extraction.

DECISION 02

Next.js App Router with the Node.js runtime

The platform needs fast public pages, authenticated dashboards, server-side data access, form mutations, and external HTTP endpoints. App Router provides these in one coherent application model.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Server Components	Public pages and dashboard reads can access repositories directly without a browser-to-internal-API round trip.	Client-heavy SPA: useful only if offline behavior or highly interactive local state becomes dominant.
Server Actions	First-party form mutations gain typed server boundaries and progressive enhancement.	REST for every UI action: extra contracts and network hops with little MVP benefit.
Route Handlers	Public APIs, webhooks, and future mobile clients retain explicit HTTP semantics.	Server Actions are not suitable for external integrations.
Node.js runtime	Database clients, provider SDKs, crypto, and debugging tools work without Edge restrictions.	Edge runtime: revisit only for measured geographic latency where dependencies are compatible.
TypeScript strict mode	Shared contracts and domain states are checked before runtime; unsafe boundaries remain explicit.	Plain JavaScript would reduce setup but increase ambiguity in a multi-module domain.

Boundary rule

React coordinates presentation. Scheduling, authorization, and appointment lifecycle policy live in framework-independent domain modules so the business model can outlast the web framework.

The choice is optimized for development coherence, not framework lock-in. The adapter and domain boundaries are the escape hatch.

DECISION 03

PostgreSQL as the consistency and isolation boundary

The most important guarantees are relational: a user belongs to a tenant, staff serve specific services, appointments reference valid tenant-owned records, and active time ranges must not overlap.

Why PostgreSQL

- Transactions make appointment, event, audit, idempotency, and outbox writes succeed or fail together.
- Range types and GiST exclusion constraints express the real scheduling invariant directly.
- Row-level security provides a second tenant boundary below application code.
- Managed PostgreSQL offers mature backup, point-in-time recovery, observability, and migration tooling.

Why shared-schema multi-tenancy

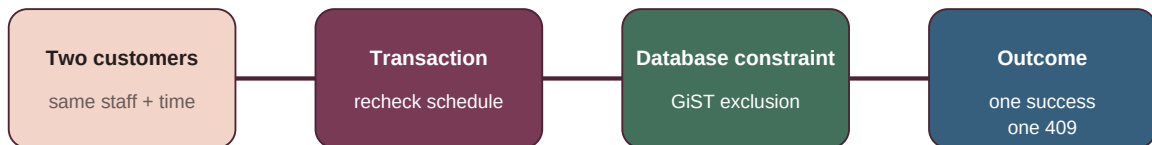
Every tenant-owned row carries an explicit `tenant_id`. The runtime sets transaction-local tenant context, and RLS restricts visibility. Composite foreign keys prevent a child row from referencing a parent in another tenant.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Shared schema + RLS	Economical for many small tenants; one migration path; efficient cross-tenant operations for audited platform tooling.	Database per tenant: revisit for regulatory isolation, very large tenants, or contractual restore requirements.
SET LOCAL tenant context	Transaction scope prevents tenant identity leaking when pooled connections are reused.	Session-wide SET is prohibited because pooled sessions outlive requests.
Application RBAC + RLS	RBAC answers what the member may do; RLS answers which tenant rows can be touched.	Either control alone leaves a larger single point of failure.

DECISION 04

The database, not the calendar UI, owns the slot

Availability changes between display and submission. The design treats displayed slots as advisory and performs the decisive check inside the write transaction.



The UI predicts availability; PostgreSQL decides who owns the slot.

Correctness remains intact under retries, multiple app instances, and simultaneous submissions.

Layered protection

- The request carries an idempotency key, so network retries do not create duplicate appointments.
- The transaction reloads current service, staff, schedule, exceptions, lead time, and horizon rules.
- A partial GiST exclusion constraint rejects overlapping pending or confirmed ranges for one staff member.
- The losing race becomes a stable HTTP 409 conflict with refreshed alternatives, not an internal error.
- Status updates use expected state or version checks so two managers cannot silently overwrite each other.

Why not a distributed lock?

A database constraint is closer to the data, survives application restarts, works across every app instance, and eliminates a second lock service. A distributed lock could still be lost or misconfigured while the database accepts an invalid overlap.

DECISION 05

Typed data access with explicit SQL escape hatches

Drizzle provides a typed schema and query layer, while reviewed SQL migrations retain access to PostgreSQL-specific controls that generic abstractions cannot safely hide.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Drizzle ORM	Small abstraction, TypeScript inference, visible SQL-shaped concepts, and controlled migration generation.	Prisma: productive, but a heavier abstraction for RLS and exclusion-constraint-heavy work.
Reviewed SQL migrations	RLS policies, SECURITY DEFINER functions, composite tenant constraints, and GiST exclusions stay explicit.	ORM-only migrations risk omitting the database features carrying the core guarantees.
Forward-only releases	Production history is auditable and compatible with expand/migrate/contract rollouts.	Editing applied migrations makes environments diverge.

Data conventions and their rationale

1	UUID identifiers	Globally safe across environments and services; not used as customer-facing secrets.
2	Integer money	Minor units plus ISO currency avoid floating-point errors and retain currency meaning.
3	UTC instants	timestampz stores an unambiguous moment; IANA timezone names preserve display and schedule context.
4	Immutable snapshots	Appointments retain service name, duration, price, currency, and timezone after catalog edits.
5	Soft archival	Historical references remain valid while inactive catalog items disappear from future booking.

JSONB is intentionally limited

Flexible metadata belongs in JSONB; identity, money, time, state, and relationships remain relational so constraints and query plans stay reliable.

DECISION 06

Managed identity, local authorization, explicit validation

Authentication, authorization, and validation solve different problems. The architecture keeps them separate so a successful sign-in never implies permission to act on a tenant.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Managed OIDC	Avoids storing passwords and delegates mature MFA, recovery, and session security capabilities.	Self-hosted identity: revisit for data residency, enterprise federation, or provider economics.
Local user mapping	Stable issuer/subject pairs map to memberships without making domain code vendor-specific.	Provider user IDs used everywhere would create deep lock-in.
Membership RBAC	Owner, manager, and staff permissions match the MVP operating model and are easy to audit.	Fine-grained ABAC: revisit when tenants need custom roles or location-level policies.
Zod at boundaries	Runtime schemas validate browser, webhook, URL, and environment input before domain use.	TypeScript alone disappears at runtime and cannot validate untrusted data.
Signed customer links	Customers manage one appointment without an account; only a scoped expiring token hash is stored.	Customer accounts: revisit when cross-business history or loyalty features justify friction.

Defense in depth

Authentication establishes identity. Membership establishes tenant context. Policy establishes the allowed action. RLS limits the rows. Validation protects the boundary. Audit records the outcome.

DECISION 07

Transactional outbox for notifications and retries

Email, SMS, and webhooks are slow and unreliable compared with a database transaction. They must not determine whether a valid booking exists.

The failure being avoided

If the application commits an appointment and crashes before sending email, the customer hears nothing. If it sends first and the database later rolls back, the customer receives a confirmation for a booking that does not exist. A distributed transaction with an email provider is not available.

1	Commit	Appointment and outbox event are written in the same PostgreSQL transaction.
2	Claim	A worker claims ready events with FOR UPDATE SKIP LOCKED and a recoverable lease.
3	Deliver	Provider adapters use deterministic idempotency keys and safe event snapshots.
4	Retry	Transient failures back off; exhausted events become visible dead letters.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Database-pollled outbox	No extra infrastructure is needed for MVP; atomicity is straightforward.	Durable queue: introduce when outbox volume, latency, or multiple consumer groups justify it.
At-least-once delivery	Recoverability is favored over silently losing a message.	Exactly-once delivery is not realistic across external providers; make consumers idempotent instead.
Provider adapters	Templates and domain events remain stable when email or messaging vendors change.	Direct SDK calls in use cases would couple core logic to external availability.

DECISION 08

Cache content; calculate commitments

The platform separates information that may safely be stale from information that grants scarce staff capacity.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Tenant-tagged public cache	Published profiles and catalogs change infrequently and benefit from fast global delivery.	Invalidate after publication changes; never key only by a generic route.
Dynamic availability	Slots depend on current appointments, lead time, schedules, and exceptions.	Only use short/private caching when every relevant version is part of the key.
Direct server reads	Server Components avoid an internal HTTP hop and keep secrets on the server.	Client fetching remains appropriate for live interactive refresh where needed.
Purpose-built indexes	Tenant/location/time, tenant/staff/time, and tenant/status/time reflect the actual access paths.	Add or change indexes from measured query plans, not speculation.

Observability is part of correctness

- A correlation ID connects request logs, traces, audit records, database context, and outbox events.
- Booking conflicts are a product metric and a concurrency signal, not merely errors.
- Outbox lag and dead letters reveal customer communication failures before support tickets do.
- Logs avoid raw tenant slugs and customer PII; diagnosis must not create a second data leak.

Operational posture

The target is not maximum telemetry. It is enough structured evidence to answer: which request failed, for which tenant context, at which boundary, and whether retry is safe.

DECISION 09

Test the invariant at the layer that enforces it

A green component test cannot prove tenant isolation or race safety. The test strategy mirrors the architecture's layers and uses the real PostgreSQL engine where database behavior matters.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Unit tests	Status transitions, permissions, timezone math, and pricing rules remain fast and deterministic.	Do not mock the database to claim RLS or constraint behavior.
Property tests	Generated schedule combinations expose DST, interval, and exception edge cases humans overlook.	Example-only tests leave a large temporal state space uncovered.
PostgreSQL integration	RLS, composite FKs, transactions, idempotency, and exclusion conflicts are verified where they execute.	In-memory databases are not faithful substitutes for these features.
Parallel booking tests	Two simultaneous attempts must produce exactly one active appointment.	Sequential tests cannot expose the race the constraint exists to stop.
End-to-end tests	A small set proves onboarding, publication, booking, confirmation, and cancellation across boundaries.	Avoid pushing every permutation into slow browser tests.

Delivery discipline

Forward-only migrations, separate migration credentials, preview environments without production PII, and expand/migrate/contract releases reduce the chance that deployment mechanics violate the model.

EVOLUTION MAP

Alternatives were deferred, not dismissed

Good architecture defines the evidence that would change the decision. These triggers keep the design adaptable without paying speculative complexity upfront.

DECISION	WHY THIS FITS NOW	ALTERNATIVE / REVISIT TRIGGER
Microservices	Not justified for the MVP's team size and cross-module transactions.	Revisit with independent teams, incompatible scaling profiles, or deployment isolation requirements.
Database per tenant	Operationally expensive for many small tenants.	Revisit for contractual isolation, tenant-specific residency, or very large customers.
Edge runtime	Node.js compatibility is more valuable than speculative latency gains.	Revisit after measuring geographic latency on routes that do not need Node-only dependencies.
Managed queue	The database outbox is adequate at initial volume.	Revisit when polling load, delivery latency, replay tooling, or consumer fan-out becomes material.
Customer accounts	Anonymous booking reduces friction.	Revisit for loyalty, cross-tenant marketplace history, saved preferences, or subscriptions.
Auto-confirmation	Business confirmation reflects the initial operating model.	Revisit after pilot data shows schedule reliability and tenants request instant booking.
Payments	Deposits widen compliance and support scope before the booking loop is proven.	Revisit when no-show data and willingness-to-pay justify payment operations.

Decision governance

Record vendor selections and irreversible policy choices as ADRs. Every ADR should state context, decision, consequences, owner, date, and a measurable revisit condition.

CLOSING ASSESSMENT

A strong foundation with intentionally limited ambition

The proposed architecture is a fit for Chairly's current stage because it protects the risks that could invalidate the product while preserving speed of learning.

What the design guarantees when implemented correctly

- Tenant-owned data is isolated by trusted context, application policy, composite relationships, and database RLS.
- At most one pending or confirmed appointment can reserve a staff member for an overlapping time range.
- Bookings, history, audit, idempotency, and notification intent commit atomically.
- Mutable catalog changes cannot rewrite historical appointment meaning.
- Provider outages degrade communication, not the integrity of the booking record.
- Vendor-specific services can change without rewriting the scheduling domain.

The architecture promise

Keep the core small enough to understand, make the critical rules impossible to bypass accidentally, and evolve only when production evidence makes the next layer of complexity worthwhile.

Source basis

This rationale interprets the current PRD.md, Architecture.md, AGENTS.md, and initial database schema in the Blacksmithexp repository. It explains the reasoning behind the proposed design; it does not replace formal ADRs or implementation validation.

CHAIRLY / PROPOSED MVP ARCHITECTURE